

Django REST Framework

Todo API Interview

Easy to intermediate - 10 questions with answers and practical fixes

Section	Questions	Points
Debugging	1-3	30
Code Writing	4-6	45
General Knowledge	7-8	15
Code Output / Tracing	9-10	10
Total	10	100

Backend project

C:/Users/User/Desktop/Interview Question/todo-api/TODO_TEST

This combined edition is for the interviewer. Each question is immediately followed by the expected answer and a concrete repair or implementation.

Section 1

Debugging

1. The view expects POST, but request.http sends GET

QUESTION

The view implements `post()`, but the test request uses GET `http://127.0.0.1:8000/api/todos/create/` and includes a JSON body. The response is 405 Method Not Allowed. Explain the cause and correct the request in `request.http`.

ANSWER

APIView dispatches the request according to its HTTP method. A GET request looks for `get()`, but `CreateTodoView` only defines `post()`. A request body does not change GET into POST, so DRF correctly returns 405 and normally includes `Allow: POST, OPTIONS`.

HOW TO FIX IT

Change the first word in `request.http` from GET to POST. Keep the JSON content type and place a blank line between the headers and body. The Django view does not need to change.

```
# request.http
POST http://127.0.0.1:8000/api/todos/create/
Content-Type: application/json

{
  "title": "Prepare report",
  "description": "Finish before Friday",
  "completed": false
}
```

2. The view returns a dictionary instead of a Response

QUESTION

ListTodosView builds the correct data but finishes with `return {'count': len(todo_list), 'todos': todo_list}`. DRF reports that the view must return a Response or HttpResponse object. Explain the error and fix the return value.

ANSWER

A Python dictionary is only data; it is not an HTTP response. APIView expects the handler to return a DRF Response, Django HttpResponse, or StreamingHttpResponse. DRF's Response class also handles content negotiation and converts the data to JSON for the client.

HOW TO FIX IT

Wrap the dictionary in `Response(...)` and provide the appropriate status code.

```
# Wrong
return {
    "count": len(todo_list),
    "todos": todo_list,
}

# Correct
return Response(
    {
        "count": len(todo_list),
        "todos": todo_list,
    },
    status=status.HTTP_200_OK,
)
```

3. The URL parameter name does not match the view

QUESTION

The URL uses `<int:todo_id>`, but the view method is defined as `def get(self, request, id)`. A detail request raises `TypeError: get() got an unexpected keyword argument 'todo_id'`. Find the mismatch and fix it.

ANSWER

Django extracts the value using the converter name and calls the view with `todo_id=...`. Python cannot pass that named argument into a method that only accepts an argument named `id`. The names must match exactly.

HOW TO FIX IT

Use `todo_id` in both the URL and method signature. The existing lookup can then use the same variable.

```
# todos/urls.py
path(
    "todos/<int:todo_id>/",
    GetTodoView.as_view(),
    name="todo-detail",
)

# todos/views.py
class GetTodoView(APIView):
    def get(self, request, todo_id):
        todo = get_object_or_404(Todo, id=todo_id)
        return Response(
            {"id": todo.id, "title": todo.title},
            status=status.HTTP_200_OK,
        )
```

Section 2

Code Writing

4. Implement soft delete

QUESTION

Modify the Todo model and DeleteTodoView so DELETE preserves the row, records whether and when it was deleted, uses Django timezone utilities, and returns 204 with an empty body. Show the migration commands and explain how normal list/detail queries must change.

ANSWER

Add a Boolean deletion flag and nullable deletion timestamp. The delete endpoint updates those fields instead of calling `delete()`. Active API queries must explicitly exclude deleted rows; otherwise the operation is only hidden at the database level, not from users.

HOW TO FIX IT

Implement the fields and endpoint below. Using `update_fields` makes the write explicit. Filtering by `is_deleted=False` also makes a second DELETE return 404, which is a defensible idempotent-resource behavior.

```
# todos/models.py
from django.db import models

class Todo(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    completed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    is_deleted = models.BooleanField(default=False, db_index=True)
    deleted_at = models.DateTimeField(null=True, blank=True)

# todos/views.py
from django.shortcuts import get_object_or_404
from django.utils import timezone
from rest_framework import status
from rest_framework.response import Response
from rest_framework.views import APIView

class DeleteTodoView(APIView):
    def delete(self, request, todo_id):
        todo = get_object_or_404(Todo, id=todo_id, is_deleted=False)
        todo.is_deleted = True
        todo.deleted_at = timezone.now()
        todo.save(update_fields=["is_deleted", "deleted_at"])
        return Response(status=status.HTTP_204_NO_CONTENT)

# Normal client-facing queries
todos = Todo.objects.filter(is_deleted=False).order_by("-created_at")
todo = get_object_or_404(Todo, id=todo_id, is_deleted=False)

# Run after changing the model
python manage.py makemigrations todos
python manage.py migrate
```

5. Add a mark-complete endpoint

QUESTION

Create POST `/api/todos/<todo_id>/complete/`. It must find an active Todo, set `completed` to true, save only that field, and return the updated Todo with status 200. A missing or soft-deleted Todo must return 404.

ANSWER

Add one small APIView and one named URL. Use `get_object_or_404` with `is_deleted=False`, set the Boolean field, and use `update_fields` so the database update is explicit. Calling the endpoint again is safe because the Todo remains completed.

HOW TO FIX IT

Implement the action directly with Django ORM and build the response dictionary explicitly.

```
# todos/views.py
class CompleteTodoView(APIView):
    def post(self, request, todo_id):
        todo = get_object_or_404(
            Todo,
            id=todo_id,
            is_deleted=False,
        )
        todo.completed = True
        todo.save(update_fields=["completed"])

        return Response(
            {
                "id": todo.id,
                "title": todo.title,
                "completed": todo.completed,
            },
            status=status.HTTP_200_OK,
        )

# todos/urls.py
path(
    "todos/<int:todo_id>/complete/",
    CompleteTodoView.as_view(),
    name="todo-complete",
)
```

6. Add filtering to the list endpoint

QUESTION

Support `completed=true/false` and a case-insensitive search across title and description. Reject invalid completed values with 400, allow filters to combine, and preserve newest-first ordering.

ANSWER

Read query parameters as strings, validate the Boolean spelling, and build one lazy ORM `QuerySet`. Use a `Q` expression to OR the title and description search conditions. Applying both filter calls naturally combines them with AND.

HOW TO FIX IT

Filter in the database before building the response; do not fetch every row and filter a Python list.

```
from django.db.models import Q

class ListTodosView(APIView):
    def get(self, request):
        todos = Todo.objects.filter(is_deleted=False)

        completed = request.query_params.get("completed")
        if completed is not None:
            normalized = completed.lower()
            if normalized not in {"true", "false"}:
                return Response(
                    {"completed": ["Use 'true' or 'false'."]},
                    status=status.HTTP_400_BAD_REQUEST,
                )
            todos = todos.filter(completed=(normalized == "true"))

        search = request.query_params.get("search")
        if search:
            todos = todos.filter(
                Q(title__icontains=search) |
                Q(description__icontains=search)
            )

        todos = todos.order_by("-created_at")
        todo_list = [
            {
                "id": todo.id,
                "title": todo.title,
                "description": todo.description,
                "completed": todo.completed,
            }
            for todo in todos
        ]
        return Response(
            {"count": len(todo_list), "todos": todo_list},
            status=status.HTTP_200_OK,
        )
```

Section 3

General Knowledge

7. Correct the response status codes

QUESTION

A create request currently returns 200, a successful delete with no body returns 200, invalid input returns 500, and a missing Todo returns 400. Replace each with the correct HTTP status code and briefly explain what the code tells the client.

ANSWER

Successful creation should return 201 Created. A successful deletion with no body should return 204 No Content. Invalid client input should return 400 Bad Request. A Todo that does not exist should return 404 Not Found. These codes let clients distinguish success, their own input errors, and missing resources without parsing message text.

HOW TO FIX IT

Use DRF's named status constants so the meaning remains obvious during code review.

```
status.HTTP_201_CREATED      # successful create
status.HTTP_204_NO_CONTENT   # successful delete, no body
status.HTTP_400_BAD_REQUEST  # invalid request data
status.HTTP_404_NOT_FOUND    # Todo does not exist
```

8. Choose PUT or PATCH for a partial update

QUESTION

A client only wants to change completed and leave title and description unchanged. The current endpoint is named PUT but ignores fields that are not supplied. Should this request use PUT or PATCH? Explain the choice and correct the view method.

ANSWER

PATCH is the clearer choice because the client sends only the field it wants to change. Omitted fields stay unchanged. PUT conventionally represents a full replacement and should require the complete writable representation. Both methods should be idempotent for these Todo updates.

HOW TO FIX IT

The current update endpoint uses PUT but behaves like PATCH because it ignores omitted fields. Either expose it as PATCH and update only supplied fields, or make PUT require the complete writable representation. The current delete endpoint should return 204 with no message body.

```
class UpdateTodoView(APIView):
    def patch(self, request, todo_id):
        todo = get_object_or_404(Todo, id=todo_id, is_deleted=False)
        allowed_fields = {"title", "description", "completed"}

        for field in allowed_fields:
            if field in request.data:
                setattr(todo, field, request.data[field])

        todo.save()
        return Response({
            "id": todo.id,
            "title": todo.title,
            "description": todo.description,
            "completed": todo.completed,
        })
```

Section 4

Code Output / Tracing

9. Trace the current update view

QUESTION

Todo 7 starts as title='Write tests', description='Cover the API', completed=False. The client sends {'completed': true, 'description': null} to the current PUT view. State the local values, changed fields, final stored values, and whether the endpoint is strict PUT.

ANSWER

The local values are title=None, description=None, and completed=True. Because every assignment is guarded by `is not None`, only completed is changed. The final row remains title='Write tests', description='Cover the API', completed=True. The endpoint behaves like PATCH, not strict PUT; it also cannot intentionally set a nullable field to null.

HOW TO FIX IT

Use `field in request.data` to distinguish a missing field from a field explicitly sent as null. Validate each supplied value before assigning it. Use PATCH for this partial-update behavior; a strict PUT should require every writable field.

```
# Manual partial update example
if "title" in request.data:
    title = request.data["title"]
    if not isinstance(title, str) or not title.strip():
        return Response(
            {"error": "title must be a non-empty string."},
            status=status.HTTP_400_BAD_REQUEST,
        )
    todo.title = title.strip()

if "completed" in request.data:
    completed = request.data["completed"]
    if not isinstance(completed, bool):
        return Response(
            {"error": "completed must be true or false."},
            status=status.HTTP_400_BAD_REQUEST,
        )
    todo.completed = completed

todo.save()
```

10. Trace lazy QuerySet evaluation and caching

QUESTION

There are initially two incomplete Todos. A QuerySet is created; `qs.count()` and `len(list(qs))` are printed; a third incomplete Todo is inserted; then `qs.count()`, `len(qs)`, and a fresh QuerySet count are printed. State all five outputs and explain the cache.

ANSWER

The output is 2, 2, 2, 2, 3. The first count performs a database COUNT without populating the result cache. Converting `qs` to a list evaluates it and caches the two model instances. After the insert, `count()` and `len(qs)` reuse that populated cache and still see two. A newly constructed QuerySet has no old cache and sees all three rows.

HOW TO FIX IT

Create a fresh QuerySet when current database state is required, or avoid retaining an evaluated QuerySet across writes. This is normal lazy-evaluation behavior, not a database consistency bug.

```
# Fresh query after the write
Todo.objects.create(
    title="Third todo",
    description="",
    completed=False,
)

fresh_count = Todo.objects.filter(completed=False).count() # 3
```

End of interview guide